

Launch Estimate — Effort, Timeline & Cost

GamerGain / PlayEarning Nexus — PWA (web) + Android + iOS

Prepared July 24, 2026. This estimate is grounded in the actual codebase and the launch docs ([DEVELOPER-HANDOFF-BRIEF.md](#) , [LAUNCH-UNDER-100-HOURS.md](#) , [CONFIG-AND-SECRETS.md](#) , [AWS-SCALING-ARCHITECTURE.md](#)). Dollar figures are ranges, not quotes — labor depends on the developer's rate and LLM/usage costs depend on real engagement. Where a number is a projection rather than a measured result, it says so.

The short version

You are **not building an app** — you are **configuring, deploying, and shipping one that is already written**. The frontend (208-page React PWA), the Deno backend (526 function routes, 239-table Postgres schema), auth, the agent system, the payout/economy logic, and the security hardening are done and on GitHub. What remains is standing it up on live infrastructure, wiring your own provider keys, testing money flows end-to-end, and submitting the mobile apps.

Headline	Lean path (Android first)	Full path (Android + iOS + load test)
Developer effort	~70-95 hours	~95-125 hours
Calendar to web live	~2-3 weeks	~2-3 weeks
Calendar to apps live	~3-4 weeks (Android)	~4-6 weeks (both stores)
One-time cost (typical)	~\$6,000-12,000	~\$9,000-17,000
Monthly cost at launch (low traffic)	~\$100-700/mo	same
Monthly cost at 100k users	~\$3,000-8,000/mo	same

The single biggest variable in both the one-time and the monthly number is explained below: developer hourly rate for the one-time cost, and **LLM/AI usage** for the monthly cost.

1. What "launch" includes (scope)

Launching means all three surfaces live and working end-to-end:

- **Web PWA** — the React app deployed to a domain with HTTPS, installable, talking to the live backend.
- **Backend + database** — the Deno service running on a host, connected to a managed PostgreSQL database with the schema loaded, the cron scheduler on, and every integration key wired.
- **Native apps** — the Capacitor wrappers for **Android** (Play Store) and **iOS** (App Store), submitted and approved.

It also includes the non-code prerequisites: third-party accounts and keys, legal pages at public URLs, and passing each store's review.

2. Why the effort is measured in dozens (not hundreds) of hours

A large amount of work that a developer would normally bill for was already completed, which is why the remaining number is small. From [LAUNCH-UNDER-100-HOURS.md](#) , the pre-built work would otherwise have cost roughly **105-160 developer hours**: the full Base44 removal, the human-in-the-loop oversight gate on all money/fraud functions, per-agent model pinning and cost caps, the autonomous agent scheduling, the closed-loop payout enforcement, the server-side tamper-proof economy, the store markup/buy-credit flow, and a verified-green frontend build. On top of that, a **full static code audit** (July 24) found and fixed a SQL-injection vector, an admin auth-bypass, and several runtime bugs — de-risking the deployment further.

The remaining work is therefore **configuration, deployment, and testing**, not feature development.

3. Developer effort — hour breakdown

These are the tasks a developer still performs, from the launch plan. One intermediate full-stack developer, working focused.

#	Work	Hours
1	Accounts & API keys (Postgres host, OpenAI/Anthropic, SendGrid, S3, Stripe/PayPal live, Twilio, BitLabs)	8-12
2	Deploy backend (container + managed Postgres + load schema.sql) + scheduler process	10-14
3	Pre-deploy validation (<code>npm run build</code> ✓ already proven, <code>deno check</code> on Linux) + fix anything surfaced	3-6
4	Deploy frontend (Amplify or S3+CloudFront, SPA fallback, domain + SSL)	4-8
5	Payments end-to-end, sandbox → live: inbound (Stripe/PayPal, buy-credit) + partner payouts via the approval queue	10-15
6	Survey loop live test: BitLabs postback → credit → signal ingest → agent trigger → oversight queue	6-10
7	Mobile wrapper (Capacitor Android) + store submission prep	15-22
8	QA pass + fixes (auth, store, referrals, legal pages)	10-15
	Subtotal (web + Android)	66-102
9	iOS archive, App Store Connect, review iterations (needs a Mac)	+6-12
10	Full load test + resize from real numbers (defer until traffic justifies)	+10-15

Lean path (Android first, defer iOS and the load test — both recommended for a first launch):
~**70-95 hours**.

Full path (Android + iOS + load test up front): ~**95-125 hours**.

4. One-time costs

4a. Developer labor (the dominant one-time cost)

Labor = hours × rate, and the rate varies widely by who you hire:

Hours ↓ / Rate →	\$50/hr (budget/offshore)	\$90/hr (mid freelance)	\$140/hr (senior/agency)
70 h (lean, low end)	\$3,500	\$6,300	\$9,800
95 h (lean, high end)	\$4,750	\$8,550	\$13,300
125 h (full path)	\$6,250	\$11,250	\$17,500

Most likely for a mid-rate freelancer doing the lean path: ~**\$6,000-9,000**. Add iOS + load test and it moves toward **\$10,000-14,000**.

4b. Fixed one-time / first-year fees

Item	Cost	Notes
Google Play Console	\$25	one-time, lifetime
Apple Developer Program	\$99/yr	required for iOS; renews yearly
Domain name	~\$12-15/yr	e.g. gamergain.app
Legal review (recommended)	\$500-2,500	privacy/terms + earn-money/skill-vs-gambling review; templates are provided, a lawyer sign-off is the cost
Mac for iOS builds (only if you don't own one)	\$0 / ~\$600-1,000 / ~\$60-120·mo	own a Mac / buy a Mac mini / rent a cloud Mac for the build window

Typical all-in one-time (lean path, own a Mac or Android-first): ~\$6,000-12,000.
Full path incl. iOS + legal review: ~\$9,000-17,000.

5. Recurring monthly costs

Two very different worlds: **early launch (low traffic)** and **scale (100k users)**. The app runs on your own provider keys now (it used to run free on Base44's), so **AI/LLM usage is the real cost driver** and grows with engagement, not infrastructure.

5a. Early launch (hundreds to low-thousands of users)

Minimal, mostly free-tier-friendly sizing on managed services:

Line	Typical monthly
Managed PostgreSQL (Neon/Supabase/RDS small)	\$0-70
Backend container host (Render/Railway/Fly, web + worker)	\$14-50
Frontend hosting (Amplify/CloudFront/Netlify)	\$0-20
Redis cache (optional early)	\$0-15
Email (SendGrid free → paid tier)	\$0-20
S3 storage/transfer	\$1-10
Infra subtotal	~\$20-185/mo
AI / LLM usage (variable — the big one)	~\$50-500/mo
SMS (Twilio, per message)	usage-based
Realistic early all-in	~\$100-700/mo

5b. At scale (~100,000 registered users, per [AWS-SCALING-ARCHITECTURE.md](#))

This is an order-of-magnitude **projection** from the architecture, not a measured result — the load test is what turns it into a real number.

Line	Monthly
RDS Postgres (Multi-AZ r6g.xlarge + replica)	\$700-1,200
ECS Fargate (web + worker, auto-scaled)	\$300-1,200
ElastiCache Redis (2 nodes)	\$200-500
ALB + CloudFront + S3 + data transfer	\$150-500
Infra floor at scale	~\$1,500-3,500/mo

Provider usage (LLM/SMS/email) — often the largest line	usage-based, frequently \$1,500-5,000+/mo
All-in at 100k	~\$3,000-8,000+/mo

5c. Transaction fees (pass-through, not fixed)

Card/PayPal processing is roughly **2.9% + \$0.30 per transaction**. This scales with revenue, not a line you pay up front. Because the economy is closed-loop (users earn store credit; only partners get cash), your cash-out volume is bounded, which helps.

Cost controls already built in: every one of the 76 agents has a **pinned model and a daily USD cap**, and slow AI/email/SMS work is designed to run through a queue — both are there specifically to keep the LLM bill predictable. Caching the prize-pool and leaderboard reads (called out in the scaling doc) is the other big lever.

6. Timeline (calendar, not just hours)

Hours are work; calendar includes waiting on other people. A single developer at ~25-30 focused hours/week:

Milestone	Lean path	Notes
Backend + web PWA live	~2-3 weeks	Phases 1-6 + QA; gated mostly by key approvals
Android in Play Store	~3-4 weeks	Google review is typically 1-3 days but can be longer
iOS in App Store	~4-6 weeks	Mac required; earn-money apps draw extra review and often a rejection round or two
Legal review	runs in parallel	not developer time; start it early

External waits you don't control: live-payment key approvals (Stripe/PayPal), app-store review, and lawyer turnaround. Budget them as calendar, not labor.

7. Key assumptions & risks

- **You supply the accounts and keys.** Every required env var is enumerated in `CONFIG-AND-SECRETS.md`; the developer wires them, they don't hunt for them. Don't pay dev hours for signup forms.
- **iOS requires a Mac** (Xcode). If you don't have one, that's an added cost or a reason to launch Android-first.
- **Earn-money app = extra app-store scrutiny.** Present it as a real app with a demo login and frame the prize pool as skill/merit-based, not gambling. Budget for a possible review rejection + resubmit.
- **LLM cost is the monthly unknown.** It tracks engagement. Start conservative, watch the per-agent caps, and raise limits as revenue justifies. This is the number most likely to surprise you — model it against real usage in the first month.
- **The 100k cost figure is a projection.** Run the load test (`LOAD-TEST-PLAN.md`) before you rely on it; resize from measured numbers.
- **Defer what you can.** Android-first (iOS a few weeks later) and launching on the built-in queue (swap to SQS when scale demands) keep the first launch inside the lean estimate.

8. Bottom line

To get **live on web + Android** with everything working, plan for roughly **70-95 developer hours** (~\$6,000-12,000 at typical rates), a **couple hundred dollars** of fixed fees, and **~\$100-700/mo** to run at launch traffic. Adding **iOS** and an up-front **load test** pushes effort to **~95-125 hours** and one-time cost toward **\$10,000-17,000**. Running costs stay modest until you grow; at

100,000 users budget ~\$3,000-8,000/mo, with AI/LLM usage — not servers — usually the largest line.

All figures are planning ranges based on the current codebase and launch docs. Labor varies with the developer's rate; monthly cost varies with real usage. The scale figures are architectural projections to be confirmed by a load test.